

OPTIONAL CHAINING (?.) & NULLISH COALESCING (??) IN JAVASCRIPT

JavaScript introduced Optional Chaining (?.) and Nullish Coalescing (??) in ES2020 to handle deeply nested objects and nullish values more effectively. Today, let's break down:

- ✓ What is Optional Chaining (?.)
- ✓ What is Nullish Coalescing (??)
- ✓ How they improve code readability & prevent errors

I. Optional Chaining (?.)

Optional Chaining (?.) safely accesses nested properties without causing errors if a property is null or undefined.

✓ Without Optional Chaining (✗ Risk of Errors)

```
let user = {};  
console.log(user.profile.name); // ✗  
ERROR: Cannot read properties of  
undefined
```

✓ With Optional Chaining (✓ Safe Access)

```
let user = {};  
console.log(user.profile?.name); // ✓  
Output: undefined (No error!)
```

- ◆ ?. checks if profile exists before accessing name.

✓ Example: Deeply Nested Objects

```
let user = {  
  profile: {  
    name: "John",  
    address: { city: "New York" }  
  }  
};
```

```
console.log(user.profile?.name); // ✓  
"John"  
console.log(user.profile?.address?.city);  
// ✓ "New York"  
console.log(user.profile?.address?.zipCode  
); // ✓ undefined (No error!)
```

- ◆ Without ?., checking address.zipCode could crash the script if address was undefined.

✓ Optional Chaining with Functions & Arrays

- ◆ Accessing methods safely

```
let user = { greet: () => "Hello" };  
console.log(user.greet?.()); // ✓ "Hello"
```

```
let user2 = {};  
console.log(user2.greet?.()); // ✓ undefined  
(No error!)
```

◆ Accessing array elements safely

```
let users = [{ name: "Alice" }];  
console.log(users[0]?.name); // ✓  
"Alice"  
console.log(users[1]?.name); // ✓  
undefined (No error!)
```

2. Nullish Coalescing Operator (??)

The Nullish Coalescing Operator (??) assigns a default value only if the left-hand side is null or undefined, unlike ||, which treats falsy values (like 0 or "") as false.

✓ Using || (Can Cause Issues)

```
let value = 0 || "Default"; // ✗ 0 is  
falsy, so "Default" is used  
console.log(value); // Output: Default
```

- ◆ Problem: If `value = 0`, it wrongly assigns "Default" because `0` is falsy.

✓ Using ?? (Correct Approach)

```
let value = 0 ?? "Default"; // ✓ 0 is  
NOT null or undefined, so it remains 0  
console.log(value); // Output: 0
```

- ◆ ?? only assigns "Default" when the left-hand side is null or undefined, not other falsy values.

✓ Example: Handling Default Values

```
let userName = null;  
console.log(userName ?? "Guest"); // ✓  
"Guest"
```

```
let age = 0;  
console.log(age ?? 18); // ✓ 0 (correct,  
because 0 is a valid number)
```

✓ Optional Chaining (?.) prevents errors by safely accessing properties/methods

✓ Nullish Coalescing (??) assigns a default value only for null or undefined, not other falsy values like 0 or ""